

Шпаргалка по pandas

Методы и приёмы по Kaggle Learn: Pandas и парам «Анализ данных». **df** — DataFrame (таблица), **s** — Series (один столбец).

1. Импорт и создание

Команда	Что делает	Пример / результат
<code>import pandas as pd</code>	подключить библиотеку	<code>import numpy as np</code> (для чисел)
<code>pd.DataFrame({'a': [1, 2], 'b': [3, 4]})</code>	таблица из словаря: ключ — столбец, список — значения сверху вниз	списки одной длины
<code>pd.DataFrame(..., index=['x', 'y'])</code>	то же, но свои названия строк	без <code>index=</code> строки с 0
<code>pd.Series([1, 2, 3], name='Имя')</code>	один столбец	значения — список, не словарь

2. Чтение и запись файлов

Команда	Что делает	Пример / результат
<code>pd.read_csv('путь.csv')</code>	прочитать CSV в DataFrame	путь — строка, в кавычках
<code>pd.read_csv('f.csv', index_col=0)</code>	первый столбец файла — индекс	лечит столбец Unnamed: 0
<code>pd.read_csv('f.csv', sep=';')</code>	другой разделитель	csv из Excel часто с ';'
<code>pd.read_excel('f.xlsx')</code>	прочитать Excel	нужен пакет <code>openpyxl</code>
<code>df.to_csv('f.csv', index=False)</code>	сохранить в CSV без столбца индекса	без <code>index=False</code> появится Unnamed: 0

3. Осмотреться в данных

Команда	Что делает	Пример / результат
<code>df.shape</code>	размер (строк, столбцов)	без скобок — это не функция
<code>df.head()</code> / <code>df.head(10)</code>	первые 5 / 10 строк	<code>df.tail()</code> — последние
<code>df.info()</code>	столбцы, типы, число непустых	быстро видно, где пропуски
<code>df.describe()</code>	count, mean, std, min, 25%, 50%, 75%, max	по числовым столбцам
<code>df.columns</code> / <code>df.index</code>	названия столбцов / индекс	<code>df.columns.tolist()</code> — список
<code>df.dtypes</code>	тип каждого столбца	int64, float64, object (строки)
<code>df.sample(5)</code>	5 случайных строк	лучше <code>head()</code> : не только начало

4. Выбор данных

Команда	Что делает	Пример / результат
<code>df['col']</code> или <code>df.col</code>	один столбец → Series	с пробелом в имени — только <code>df['my col']</code>
<code>df[['c1', 'c2']]</code>	несколько столбцов → DataFrame	двойные скобки: список внутри
<code>df.iloc[0]</code>	строка по номеру позиции	<code>iloc</code> [строки, столбцы]
<code>df.iloc[:, 0]</code>	все строки, первый столбец	':' — «всё»
<code>df.iloc[1:3, 0]</code>	строки 1 и 2 (конец не включён)	<code>iloc[-5:]</code> — последние 5
<code>df.loc[0, 'country']</code>	по названию (метке)	<code>loc[0:3]</code> — включая 3!
<code>df.loc[:, ['c1', 'c2']]</code>	все строки, нужные столбцы	
<code>df.set_index('title')</code>	сделать столбец индексом	<code>df.reset_index(drop=True)</code> — обратно с 0

Граница среза: `iloc[0:3]` → строки 0, 1, 2. `loc[0:3]` → 0, 1, 2 и 3 (включительно).

5. Фильтрация строк по условию

Команда	Что делает	Пример / результат
<code>df.loc[df['c'] == 'Italy']</code>	только строки, где условие True	<code>==</code> равно, <code>!=</code> не равно
<code>df.loc[(A) & (B)]</code>	И	каждое условие в скобках
<code>df.loc[(A) (B)]</code>	ИЛИ	
<code>df.loc[~(A)]</code>	НЕ	
<code>df.loc[df['c'].isin(['a', 'b'])]</code>	значение из списка	
<code>df.loc[df['c'].notnull()]</code>	где нет пропуска	<code>isnull()</code> — где пропуск
<code>df.query('points > 90 and country == "Italy"')</code>	условие строкой	здесь можно <code>and</code> / <code>or</code>

В pandas не `and` / `or` / `not`, а `&` / `|` / `~`, и **каждое условие в скобках**.

6. Пропуски (NaN)

Команда	Что делает	Пример / результат
<code>df.isnull().sum()</code>	сколько пропусков в каждом столбце	первое, что проверяют
<code>df.dropna()</code>	выбросить строки с пропусками	<code>dropna(subset=['c'])</code> – только по столбцу
<code>df['c'].fillna(0)</code>	заменить пропуски значением	<code>fillna(df['c'].mean())</code>
<code>df.drop_duplicates()</code>	убрать повторяющиеся строки	<code>subset=['c']</code> – по столбцу

7. Изменение данных

Команда	Что делает	Пример / результат
<code>df['new'] = 'знач'</code>	новый столбец с одним значением	если есть – перезапишется
<code>df['Age'] = 2025 - df['Year']</code>	столбец из других столбцов	работает поэлементно
<code>df.rename(columns={'old': 'new'})</code>	переименовать столбец	возвращает новую таблицу
<code>df.drop(columns=['c1', 'c2'])</code>	удалить столбцы	строки: <code>df.drop(index=[0, 1])</code>
<code>df['c'].astype(int)</code>	сменить тип	'float', 'str', 'category'
<code>pd.to_numeric(df['c'], errors='coerce')</code>	строку в число, ошибки → NaN	
<code>df.select_dtypes(include='number')</code>	только числовые столбцы	bool не входит

8. Сводные функции

Команда	Что делает	Пример / результат
<code>s.mean() / median() / max() / min()</code>	одно число	<code>sum()</code> , <code>std()</code> , <code>count()</code>
<code>s.unique()</code>	список уникальных значений	
<code>s.nunique()</code>	сколько уникальных	
<code>s.value_counts()</code>	сколько раз встречается каждое, по убыванию	<code>normalize=True</code> – доли
<code>s.mode()</code>	самое частое — это Series!	<code>s.mode()[0]</code> – само значение
<code>s.idxmax() / s.idxmin()</code>	метка индекса максимума / минимума	не само значение
<code>s.corr(s2) / df.corr()</code>	корреляция	<code>df.corr(numeric_only=True)</code>

9. map, replace, apply

Команда	Что делает	Пример / результат
<code>s.map({'red': 'красный'})</code>	заменить по словарю	нет ключа → NaN!
<code>s.replace({'red': 'красный'})</code>	заменить по словарю	нет ключа → остаётся как было
<code>s.replace(['a', 'b'], 'Rare')</code>	список значений → одно	
<code>s.apply(func)</code>	функция к каждому значению	func без скобок
<code>df.apply(func, axis=1)</code>	функция к каждой строке	<code>row['col']</code> – значения строки
<code>s.apply(lambda x: x * 2)</code>	короткая функция на месте	функция должна return

10. Сортировка

Команда	Что делает	Пример / результат
<code>df.sort_values('c')</code>	по возрастанию	<code>ascending=False</code> – по убыванию
<code>df.sort_values(['c1', 'c2'])</code>	по нескольким столбцам	<code>ascending=[True, False]</code>
<code>df.sort_index()</code>	по индексу	
<code>df.nlargest(5, 'c') / nsmallest</code>	топ-5 по столбцу	короче <code>sort_values().head()</code>

11. groupby и агрегация

Команда	Что делает	Пример / результат
<code>df.groupby('c')['v'].mean()</code>	среднее v в каждой группе c	результат – Series; индекс = группы
<code>df.groupby('c')['v'].agg(['min', 'max'])</code>	несколько функций сразу	→ DataFrame
<code>df.groupby(['a', 'b'])['v'].nunique()</code>	группы по нескольким столбцам	сколько разных v в паре (a, b)
<code>df.groupby('c').size()</code>	сколько строк в каждой группе	
<code>df.groupby('c')['v'].mean().reset_index()</code>	превратить в обычную таблицу	группа снова становится столбцом
<code>df.pivot_table(values='v', index='a', columns='b', aggfunc='mean')</code>	сводная таблица	как в Excel

12. Объединение таблиц

Команда	Что делает	Пример / результат
<code>pd.concat([df1, df2])</code>	склеить друг под другом	<code>ignore_index=True</code> ; <code>axis=1</code> – рядом
<code>df1.merge(df2, on='key')</code>	объединить по общему столбцу (как JOIN)	<code>how='inner'</code> по умолчанию
<code>df1.merge(df2, on='key', how='left')</code>	оставить все строки левой	<code>right</code> , <code>outer</code> – другие виды

13. Строки и даты

Команда	Что делает	Пример / результат
<code>s.str.lower()</code> / <code>upper()</code>	регистр	для всего столбца сразу
<code>s.str.contains('abc')</code>	True/False: есть ли подстрока	<code>na=False</code> , <code>case=False</code>
<code>s.str.replace('a', 'b')</code>	заменить часть строки	это не <code>s.replace!</code>
<code>s.str.split(' ').str[0]</code>	разбить и взять первую часть	
<code>s.str.strip()</code> / <code>s.str.len()</code>	убрать пробелы по краям / длина	
<code>pd.to_datetime(s)</code>	строки → даты	<code>format='%d.%m.%Y'</code>
<code>s.dt.year</code> / <code>.month</code> / <code>.dayofweek</code>	части даты	только после <code>to_datetime</code>

14. NumPy минимум

Команда	Что делает	Пример / результат
<code>a = np.array([1, 2, 3])</code>	массив	<code>a * 2</code> → <code>[2, 4, 6]</code> (y list – повтор!)
<code>np.arange(0, 10, 2)</code>	<code>[0, 2, 4, 6, 8]</code>	как <code>range</code> , но массив
<code>np.zeros(5)</code> / <code>np.ones(5)</code>	массив из нулей / единиц	
<code>a.mean()</code> / <code>a.sum()</code> / <code>a.max()</code>	сводные функции	<code>a.shape</code> , <code>a.dtype</code>
<code>a[a > 1]</code>	маска: только элементы > 1	как фильтрация в pandas
<code>np.where(a > 1, 'да', 'нет')</code>	значение по условию	как <code>if/else</code> для всех

15. Типичные ошибки

Что вижу	Причина	Как лечить
<code>SyntaxError</code> в <code>read_csv</code>	путь без кавычек	'путь.csv'
<code>Unnamed: 0</code>	в файле уже был индекс	<code>index_col=0</code> или <code>to_csv(index=False)</code>
<code>KeyError: 'col'</code>	нет такого столбца (опечатка, пробел)	<code>df.columns</code> , <code>strip()</code>
<code>ValueError: truth value ... ambiguous</code>	and/or вместо &/	&, и скобки
<code>SettingWithCopyWarning</code>	правка среза, а не копии	<code>df2 = df[...].copy()</code>
в столбце <code>None</code> после <code>apply</code>	функция <code>print</code> , а не <code>return</code>	поставить <code>return</code>
<code>NameError</code> после перерыва в Colab	память сброшена	Среда выполнения → Выполнить все
мода пустой выборки: <code>IndexError</code>	<code>mode()[0]</code> на пустой таблице	сначала проверить <code>sel.shape[0]</code>